

OLLAMA-REMEDIATION

Runbook: ollama embeddings on a Vulkan iGPU

Audience: the operator of the machine running the embedding backend. **Applies to:** any host where ollama serves the Lumen embedding model from a GPU driven by Vulkan — in practice an Intel iGPU (library=Vulkan, type=iGPU) with the i915 kernel driver. **Does not apply to:** hosts where ollama ps reports 100% CPU, or where inference compute reports a library other than Vulkan.

This is the *what to do about it* document. The evidence, the harness, the controlled conditions and the upstream bug links live in [OLLAMA-NAN-WEDGE.md](#) — read that if you want to know *why*; read this if you want to know *what to type*.

One-line verdict: take the iGPU out of the embedding path. Do not upgrade ollama hoping it is fixed — it is not fixed upstream, and on ALT Sisyphus there is no newer package anyway.

0. STATUS ON THIS HOST: RESOLVED

Tier 3 has been applied and verified. /etc/sysconfig/ollama carries GGML_VK_VISIBLE_DEVICES=-1, the service has been restarted, and ollama’s own journal now reports library=cpu.

Check	Before	After
inference compute ... library=	Vulkan (Intel Iris Xe, type=iGPU)	cpu
ollama ps PROCESSOR	100% GPU	100% CPU
	hundreds	0

Check	Before	After
i915 Fence expiration / GPU HANG since ollama started		
32-chunk / 18,576- char batch	repeated stale vector under HTTP 200	32 distinct vectors, 5 runs out of 5

The remaining i915 lines in a 24-hour window are pre-remediation history. `scripts/ollama-vulkan-remediation.sh --check` accounts for that explicitly: it counts faults **since the current ollama start**, and labels the 24-hour figure as history.

This runbook is now mostly a record and a re-application guide. Read §7 if you need to apply Tier 3 on another host, or if an ollama package upgrade replaces the unit or the environment file and puts you back on the corrupting path. Read §9 either way: **the fix does not repair vectors already written.**

There is a script for all of this now. Everything §7 tells you to type by hand is implemented, with verification, in `scripts/ollama-vulkan-remediation.sh`:

```
./scripts/ollama-vulkan-remediation.sh --check      # read-only;  
the default  
./scripts/ollama-vulkan-remediation.sh --apply      # sudo +  
restart, then verifies  
./scripts/ollama-vulkan-remediation.sh --verify     # library=cpu  
AND a clean batch probe  
./scripts/ollama-vulkan-remediation.sh --rollback  # undo, sudo
```

Full flag reference, exit codes and the list of things it deliberately will not do: [OPERATIONAL-SCRIPTS.md](#).

1. The symptom, as a user experiences it

You do not see “Vulkan”. You see one of these, in roughly this order of escalation:

a. Lumen prints its usage text after a failed index. This is the one that wastes the most time, because it reads like *you* mistyped the command:

```
{"error":"failed to encode response: json: unsupported value: NaN"}
```

Usage:

```
lumen index <project-path> [flags]
```

...

The command was fine. The backend returned HTTP 500, Lumen surfaced the error and then printed its help — and the help is the last thing on screen, so it is the thing you read.

b. Searches hang, then time out. This host was measured at `OLLAMA_NUM_PARALLEL=1`, i.e. one embedding request at a time — check yours with `bash scripts/ollama-tune.sh`, which reads the value off the running service rather than assuming it. At a parallelism of 1, a request that is quietly waiting out a ~20 s GPU fence timeout blocks every other client behind it. Interactive search latency goes from ~0.3 s to tens of seconds, then to nothing.

c. Searches “work” but return nonsense, or return nothing for files you know exist. This is the dangerous one and it produces **no error at all**. See §3.

c2. Nothing looks wrong at all. The worst case has no user-visible symptom whatsoever. The backend returns HTTP 200 with a **well-formed, unit-norm, 768-dimension vector** — it is simply the *previous* vector, repeated. Searches keep working. Health checks keep passing. A full forensic audit of the index passed every test it ran and declared the index TRUSTWORTHY. It was wrong: **758 vectors across 55 files** were stale duplicates. See §3, mode 4, and [INDEX-CORRUPTION-RECONCILIATION.md](#).

d. The wizard says the backend is wedged. `scripts/setup-agents-wizard.sh` catches the hard form:

```
Embedding backend is WEDGED at http://localhost:11434 - it
returns NaN for every input.
```

```
Fix: ollama stop ordis/jina-embeddings-v2-base-code (a fresh
runner clears it)
```

`ollama stop` *does* clear it — for a while. It is first aid, not treatment: the next context-filling chunk wedges it again. That is what this runbook is for.

2. Thirty-second diagnosis

One command does all of this now, including the batch probe that steps 1–5 below cannot do:

```
./scripts/ollama-vulkan-remediation.sh --check      # read-only;  
this is the default action
```

It reports the backend library (scanned from the service start timestamp, so a busy journal cannot hide it), whether the environment file carries the flag, the i915 fault count **since ollama started**, and a 32-distinct-texts batch probe that requires 32 distinct vectors back. Note that `--check` always exits 0 — read its output, or use `--verify`, which does not.

The manual equivalent, if you want to see the raw evidence. Nothing here writes, restarts, or reconfigures anything.

```
# 1. Is the embedding model being served from a GPU at all?  
ollama ps  
#    PROCESSOR column: "100% GPU" -> affected path.  "100% CPU" ->  
    you are already safe.  
  
# 2. Which compute library did ollama pick?  
journalctl -u ollama --no-pager | grep -F 'inference compute' |  
tail -1  
#    library=Vulkan + type=iGPU -> this runbook applies.  
  
# 3. Has the kernel already been abandoning GPU work?  
journalctl -k --no-pager --since "24 hours ago" | grep -c 'Fence  
expiration time out'  
journalctl -k --no-pager --since "24 hours ago" | grep -c 'GPU  
HANG'  
#    Any non-zero fence count while only embeddings ran ->  
    confirmed.  
  
# 4. Is the backend wedged right now?  
curl -s http://localhost:11434/api/embed \  
-d '{"model":"ordis/jina-embeddings-v2-base-  
code","input":"health check"}' | head -c 80  
#    healthy -> {"model":"...", "embeddings": [[0.0...  
#    wedged -> {"error":"failed to encode response: json:  
    unsupported value: NaN"}  
  
# 5. Has indexing already failed on it?  
grep -F 'unsupported value: NaN' ~/.local/share/lumen/debug.log |  
tail -3
```

Reading the result

What you saw	Verdict
100% GPU and library=Vulkan	Affected. Pick a tier below.
100% GPU, library is something else (CUDA, ROCm)	Not this defect. The tiers still work but you probably do not want them.
100% CPU	Already remediated (or the model was never offloaded). Nothing to do.
Step 4 returned NaN	Wedge d right now . <code>ollama stop <model></code> unblocks you this minute; then fix it properly.
Step 3 non-zero, step 4 healthy	Silent corruption is in progress. Step 4 passing proves nothing — see §3.

A passing health check is not a healthy index. Step 4 sends a 12-character input in a batch of one. That never trips the bug: the fault needs a large **batch** (§3). The wizard’s own round-trip probe has exactly the same blind spot, on purpose — a probe large enough to detect the fault is a probe large enough to *cause* it, and the wizard must not corrupt an index while checking it.

The operational scripts accept that cost deliberately and send a real 32-text batch, because nothing smaller can see failure mode 4. Do not run `./scripts/ollama-vulkan-remediation.sh --check` against a backend you know is on Vulkan while an index run is in flight — probe first, or fix first, but not both at once.

3. What is actually going wrong (the 90-second version)

1. A large embedding **request** arrives. Note the wording — see the correction below; the operative quantity turned out to be the size of the whole batch, not of one chunk.
2. `ggml-vulkan` issues one large compute dispatch to the iGPU.
3. The i915 driver’s fence wait expires (~20 s). The kernel logs `Fence expiration time out i915-...:ollama[pid]` and **abandons the dispatch**.
4. **ollama reads the result buffer anyway**. It was never validly written.

5. What you get back depends on what was in that buffer. **Four modes, worst last:**

#	What comes back	HTTP	Caught by a per-vector check?
1	{"error": "... unsupported value: NaN"}	500	Yes — loud
2	An all-zero vector	200	Yes — a zero-norm test sees it
3	The runner wedges: every later request returns NaN until the model is unloaded	500	Yes, but usually blamed on the caller
4	A repeated STALE vector — well-formed, 768 dims, L2 norm 1.000000	200	NO. Invisible to NaN, Inf, all-zero and norm checks alike.

1. Repeated timeouts degrade the Vulkan context until the kernel resets the device (context reset due to GPU hang). Past that point every request fails instantly — even "hi", in 0.06 s — until the runner process is destroyed.

The HTTP 500 is the tombstone, not the fault. By the time you see NaN, corrupt vectors have already been written to the index under HTTP 200. That is why every tier below is paired with a recommendation to reindex, and why “just restart it when it breaks” is not a remediation.

Correction: it is the BATCH TOTAL, not the chunk

This section originally attributed the fault to a single ~8 000-character chunk filling the model’s context. **That hypothesis was falsified.** Lumen batches **32 chunks per /api/embed request**, and inside the corrupted range on this host the *largest single chunk was 2,832*

characters — comfortably below the 4 000-char level at which zero fence timeouts were measured. The 24 requests that produced the corruption carried **16,698–31,364 characters each**.

Chunk size was never the protective factor. Any probe you build must therefore send a **batch**, not a big string — which is exactly what `scripts/ollama-vulkan-remediation.sh` and `scripts/lumen-reindex.sh` do (32 distinct texts; 32 distinct vectors required back).

Full derivation, with the per-file breakdown of all 758 affected vectors: [INDEX-CORRUPTION-RECONCILIATION.md](#).

Why mode 4 defeated a full forensic audit

[LUMEN-INDEX-INTEGRITY.md](#) audited this index and concluded it was **TRUSTWORTHY**. Every number in it is correct and reproduces exactly: 0 NaN, 0 Inf, 0 all-zero, 0 wrong-dimension, every norm inside 0.999999–1.000001.

The conclusion did not follow from the measurements. All four tests are *per-vector*, and a stale-but-well-formed vector passes all four by construction. The one check that would have caught it — aggregate distinctness — was run on blocks 1–4 only (4,096 vectors of 35,717). The corruption lived in blocks 28–29.

The lesson, and the reason `scripts/lumen-index-doctor.sh` exists:

A vector can be perfectly well-formed and still be the wrong vector. Health is not a property you can establish one vector at a time. N distinct texts must yield N distinct vectors, and nothing short of that comparison can tell you whether they did.

The settled verdict is [INDEX-CORRUPTION-RECONCILIATION.md](#). [LUMEN-INDEX-INTEGRITY.md](#) is kept unedited as a historical record of how a rigorous audit reached a wrong conclusion.

The GPU is not even buying you speed at the size that matters: at full context it “finishes” in ~20 s because it is *timing out*, not computing. The CPU takes ~70–110 s and returns a correct, bit-deterministic vector.

4. Before you choose: the index trade-off

This is the decision that actually matters, and it is not about performance.

Lumen keys each index database by (**project path, embedding model**). Evidence:

- Every index lives in `~/.local/share/lumen/<opaque-hash>/index.db`, and each database records its own identity in `project_meta`:

```
$ sqlite3 ~/.local/share/lumen/<hash>/index.db 'select key,
value from project_meta;'
vec_dimensions|768
root_hash|
3df357055b55bac731ba96837b817238d3476767ed0144d558e7358dda94641
embedding_model|ordis/jina-embeddings-v2-base-code
project_path|<ext-volume-host>/Projects/vasic
last_indexed_at|...
total_files|2413
```

- `lumen purge --help` documents the consequence in as many words:

“Each path is normalized to its git root first, then matched against the `project_path` recorded inside each index database, so switching embedding models or using custom models never leaves orphan indexes.”

Purge matches on the recorded `project_path` **because** the directory name alone cannot find every index for a project — a model switch puts the new index somewhere else. Purge was built to clean up exactly the orphan this trade-off creates.

- Vectors from two different models are not comparable. `vec_dimensions` is fixed per database and cosine similarity across two different embedding spaces is meaningless, so reuse is not merely unimplemented — it is not possible.

Therefore

	Changes LUMEN_EMBED_MODEL?	Existing index	Cost
Tier 1 (<code>options.num_gpu</code>)	no	preserved	not usable by Lumen

	Changes LUMEN_EMBED_MODEL?	Existing index	Cost
			— see §5
Tier 2 (CPU-pinned model tag)	yes	DISCARDED — orphaned, and a second index is built from zero	full reindex of every project, plus the disk of both copies until you purge
Tier 3 (GGML_VK_VISIBLE_DEVICES=-1)	no	preserved and resumable	one sudo, one service restart

Verdict: Tier 3 is the better durable fix. It keeps the model name, so it keeps the index. Tier 2 changes the model name, and a different model name means a second index built from scratch while the original is orphaned on disk. On this host that is a **113 MiB** index over **2 413 tracked files** thrown away — and thrown away *per project*, for every project you have ever indexed, not just the one in front of you.

Tier 2 remains the right answer in exactly one situation: **you cannot get sudo**. It is the only tier that survives a restart without root.

Do not reach for Tier 2 as “the safe one because it does not need a restart.” The restart is the cheap part. The reindex is the expensive part, and only Tier 2 forces it on you as a permanent, per-project cost.

The restart is safe for a running index

Tier 3 needs `systemctl restart ollama`, which will kill whatever embedding request is in flight and fail the indexing job that issued it. **That is recoverable and cheap**, because Lumen's index is incremental and resumes:

```
01:35:33 ERROR "indexing failed" project="../../../vasic"
          err="... embed error (HTTP 500): {"error":"...unsupported
value: NaN"}"
01:36:31 INFO  "indexing started" project="../../../vasic"
01:36:33 INFO  "indexing plan"      total_files=2413
files_unchanged=662 files_to_add=1553 ...
```

Observed in `~/.local/share/lumen/debug.log` on this host: a run killed mid-flight by the NaN fault, restarted a minute later, and picked up with **662 files already done** (the previous plan had recorded 526). Nothing was lost; the work already committed stayed committed.

Still, prefer to restart when nothing is indexing:

```
pgrep -af 'lumen.*index'      # empty output = nothing in flight,
                             restart freely
```

If it is not empty, either wait, or accept the interruption and re-run `lumen index <path>` afterwards. Do **not** kill `-9` the indexer to “clean up first” — let it die on its own or let it finish.

5. Tier 1 — per-request `num_gpu: 0`

What it is: every embedding request explicitly asks for zero GPU layers.

```
curl -s http://localhost:11434/api/embed -d '{
  "model": "ordis/jina-embeddings-v2-base-code",
  "input": ["...your text..."],
  "options": { "num_gpu": 0 }
}'
```

Cost: none to set up. ~3–4x slower per request at full context (~70–110 s vs a ~20 s GPU “answer” that is really a timeout). No restart, no root, no config file.

What it does NOT fix — read this before choosing it: Lumen cannot use it. Lumen exposes `LUMEN_EMBED_MODEL`, `LUMEN_BACKEND`, `LUMEN_EMBED_CTX`, `LUMEN_EMBED_DIMS`, `LUMEN_MAX_CHUNK_TOKENS`,

LUMEN_FRESHNESS_TTL, LUMEN_REINDEX_TIMEOUT and LUMEN_LOG_LEVEL — and nothing that injects options into the request body. Neither does any other client you did not write. Tier 1 protects **your own scripts and probes only**; it leaves the indexer, which is the thing generating the full-context chunks in the first place, completely exposed.

Rollback: stop sending options. There is no persistent state to undo.

6. Tier 2 — a CPU-pinned model tag

What it is: a derived model that carries `num_gpu 0` inside itself, so *any* caller gets CPU without sending options.

It already exists on this host and is validated 3/3 in OLLAMA-NAN-WEDGE.md §8. Confirm before you rely on it:

```
ollama list | grep jina-embeddings-code-cpu
ollama show jina-embeddings-code-cpu --modelfile | grep -E
    '^(FROM|PARAMETER)'
#    -> PARAMETER num_gpu 0          <- this line is the whole fix
```

If it is missing, recreate it (shares blobs with the original — no re-download, no extra disk for the weights):

```
printf
    'FROM ordis/jina-embeddings-v2-base-code\nPARAMETER
    num_gpu 0\n' > /tmp/Modelfile.cpu
ollama create jina-embeddings-code-cpu -f /tmp/Modelfile.cpu
```

Point Lumen at it — **this is the line that costs you the index**:

```
export LUMEN_EMBED_MODEL=jina-embeddings-code-cpu
    # and add it to ~/.bashrc to persist
```

Cost: the same ~3–4x CPU slowdown as Tier 1, **plus a full reindex of every project, plus** the orphaned original index sitting on disk until you purge it (§4). Agent-spawned MCP servers only inherit the variable if they are started from a shell that has it, so a half-applied Tier 2 gives you *two* live indexes and searches whose results depend on which process asked.

What it does NOT fix: anything that ignores `LUMEN_EMBED_MODEL` — another user's shell, a service unit, an MCP server launched from a session started before you exported it. Those keep using the GPU model and keep corrupting its index.

Rollback:

```
unset LUMEN_EMBED_MODEL
# and delete the line from ~/.bashrc
ollama rm jina-embeddings-code-cpu
# optional; frees the tag, keeps the shared blobs
```

Your original index is still there and still valid — that is the one upside of the orphaning. Searches go back to it the moment the variable is gone. The CPU-model index then becomes the orphan; clear it with `lumen purge <path>` (which matches by recorded `project_path` and so removes **all** indexes for that project, both models — you will reindex from zero afterwards).

7. Tier 3 — disable Vulkan service-wide (recommended)

What it is: ollama never discovers the Vulkan device, so it never offloads anything to it. The model name does not change, so **the existing index is preserved and keeps resuming**.

`OLLAMA_VULKAN=0` and `OLLAMA_LLM_LIBRARY=cpu` were both tested and both do **nothing** on this build — `OLLAMA_VULKAN` is already false in the running service and Vulkan is used anyway.

`GGML_VK_VISIBLE_DEVICES=-1` is the only variable that works.

This tier requires sudo. There is no passwordless sudo on this machine (`sudo -n true` → `sudo: a password is required`), so it cannot be automated and the wizard will never attempt it. An operator has to run it.

Already applied on this host — see [§0](#). The steps below are the re-application procedure (new host, or an ollama package upgrade that replaced the unit or the environment file).

Shortcut. `./scripts/ollama-vulkan-remediation.sh --apply` performs Steps 2–4 in one go: it appends the flag only if absent, restarts the service, waits 6 s, then asserts `library=cpu` **and** a clean 32-vector batch probe before reporting success. It is a no-op if the flag is already there. `--rollback` reverses Step 2 and re-checks. The hand-typed procedure below remains correct and is what the script does.

One difference worth knowing: the script **appends the bare line** `GGML_VK_VISIBLE_DEVICES=-1` and does not `chmod`. The heredoc in Step 2 writes the same setting with an explanatory comment header. If `/etc/sysconfig/ollama` on your host has no comments, the script wrote it.

Step 1 — confirm the seam (no changes yet)

```
systemctl show ollama -p FragmentPath -p DropInPaths -p
EnvironmentFiles
```

Expect:

```
FragmentPath=/usr/lib/systemd/system/ollama.service
DropInPaths=
EnvironmentFiles=/etc/sysconfig/ollama (ignore_errors=yes)
```

The unit **already** sources `/etc/sysconfig/ollama` optionally (`EnvironmentFile=-`), and the file does not exist yet. Creating it sets service-wide environment **without editing any unit file and without a drop-in**. Nothing packaged is modified.

Step 2 — create the file

```
sudo tee /etc/sysconfig/ollama >/dev/null <<'EOF'
# Intel iGPU + Mesa Vulkan corrupts embeddings at full context and
# wedges the
# ollama runner (i915 fence timeouts -> GPU hang). Upstream, still
# open:
#   https://github.com/ggml-org/llama.cpp/issues/18969
#   https://github.com/ggml-org/llama.cpp/issues/26044
# NOTE: OLLAMA_VULKAN=0 does NOT work on this build - it is
#       already false and
# Vulkan is used anyway. GGML_VK_VISIBLE_DEVICES=-1 is what
#       actually disables it.
# See docs/setup-agents-wizard/OLLAMA-REMEDIATION.md
GGML_VK_VISIBLE_DEVICES=-1
EOF
sudo chmod 0644 /etc/sysconfig/ollama
```

Step 3 — pick your moment, then restart

```
pgrep -af 'lumen.*index'          # ideally empty; if not, see §4
    "the restart is safe"
sudo systemctl restart ollama
```

Step 4 — verify (do not skip)

```
journalctl -u ollama --no-pager --since '2 min ago' | grep -F
'inference compute'
# must now show library=cpu, and no Vulkan device line at all.
```

```
ollama ps
# after the next embed, PROCESSOR must read "100% CPU".
```

```
journalctl -k --no-pager --since '10 min ago' | grep -c 'Fence
expiration time out'
# must stay at 0 while indexing runs.
```

If inference compute still says library=Vulkan, the environment file was not read — check for a typo in the path (/etc/sysconfig/ollama, no extension) and that systemctl show ollama -p EnvironmentFiles still lists it.

Cost: one sudo, one restart, and the same ~3–4x CPU slowdown on embeddings — now applied to everything the daemon serves, including any chat model you also run through it. If you use this ollama for generation as well, that is the real price of Tier 3, and it is the one reason to prefer Tier 2 despite the index cost.

What it does NOT fix: vectors already written. The corruption is in the index, not in the daemon — see §9.

Rollback:

```
sudo rm /etc/sysconfig/ollama # or comment the
GGML_VK_VISIBLE_DEVICES line out
sudo systemctl restart ollama
journalctl -u ollama --no-pager --since '2 min ago' | grep -F
'inference compute'
# -> library=Vulkan again
```

Rollback restores the defect. It does not damage the index, because the model name never changed.

8. What none of these tiers fix

Be explicit about the boundaries, so nobody reports these back as “the fix did not work”:

- **Vectors already in the index.** Every tier changes what happens next. Nothing repairs an all-zero — or, worse, a *stale-but-well-formed* — vector that was written last week under HTTP 200. On

this host that was 758 vectors across 55 files, and a fixed backend did not touch a single one of them. Check with `./scripts/lumen-index-doctor.sh`, then rebuild with `./scripts/lumen-reindex.sh <path> --force`. See §9.

- **The upstream bug.** It is open in `ggml/llama.cpp` with no merged fix ([#18969](#), [#26044](#)) and reported against ollama ([#13086](#), [#15248](#)). These tiers *avoid* it.
- **Upgrading ollama.** Installed 0.23.4 is the newest package in ALT Sisypheus, and the defect is unfixed upstream regardless. Upgrading is a gamble on an open bug *and* a step outside the distro. Upgrade for other reasons, on their own merits, after a tier is in place.
- **Serialisation.** `json: unsupported value: NaN` is the Go encoder refusing to encode a NaN float. It is a symptom. There is nothing to fix in any JSON layer.
- **Throughput.** At the parallelism this host was running (1), embeddings serialise no matter which compute library is in use. CPU embeddings are slower per request; on a large corpus that is hours, not minutes. Plan the reindex. Concurrency is a **separate** knob from the Vulkan fix and has its own tool — `bash scripts/ollama-tune.sh` measures this host's CPU/RAM/model facts and reports what its concurrency should be, `--apply` sets it. Do not copy a number out of this document: it was measured on one machine.
- **Other GPU workloads.** Tier 3 disables Vulkan for the whole daemon. If something else on this host needs iGPU acceleration through ollama, Tier 3 takes it away.

Partial mitigations that are *not* fixes

Listed so you do not rediscover them and mistake them for solutions:

- `ollama stop <model>` clears a wedged runner. The next full-context chunk wedges it again. First aid only.
 - **Capping chunk size** (`LUMEN_MAX_CHUNK_TOKENS`) keeps chunks below the context-fill boundary that triggers the fault, and would also cut latency. **Untested here** — the boundary is bracketed between 4 000 and 8 000 characters on one synthetic corpus, not pinned, and it moves with content. Treat as defence in depth on top of a tier, never instead of one.
 - `GGML_VK_DISABLE_F16=1` is exposed by `libggml-vulkan.so` and might fix the corruption while keeping the iGPU. **Untested.** If you try it, verify with the behavioural signature in [OLLAMA-NAN-WEDGE.md](#) §8, not with a journal grep.
-

9. After you apply a tier: check, then reindex clean

Any index built against the Vulkan path may contain zero-norm, subtly-wrong, or **stale duplicate** vectors. All of them were written under HTTP 200 and are invisible to every status-code check — and the stale ones are invisible to every *per-vector* check as well.

First find out whether you actually have a problem. Read-only, safe during an index run:

```
./scripts/lumen-index-doctor.sh /path/to/repo
#   exit 0 = healthy · 1 = corruption found · 2 = could not
    inspect
```

An incremental re-run will **not** repair anything it finds: a corrupted file still carries a non-empty content hash, so Lumen treats it as done and skips it forever. Force it:

```
./scripts/lumen-reindex.sh /path/to/repo --force
```

That wrapper refuses to start if the backend still reports library=Vulkan (exit 3), probes for stale-duplicate vectors before writing anything (exit 4 if the backend fails it), and retries around transient faults instead of dying halfway with Lumen's usage text on screen. The bare equivalent, if you prefer it, is:

```
lumen index /path/to/repo -f      # -f = force full re-index; re-
    embeds everything
```

Then confirm:

```
./scripts/lumen-index-doctor.sh /path/to/repo    # expect exit 0
```

Full flag and exit-code reference for both: [OPERATIONAL-SCRIPTS.md](#).

Budget for it. On CPU this is materially slower than the run that produced the bad data, and at the parallelism this host was measured to run (1) it cannot be sped up by throwing more clients at it. Check what YOUR host is set to, and what it could be, before you assume:

```
bash scripts/ollama-tune.sh
    # detect + recommend, changes nothing
bash scripts/ollama-tune.sh --print-commands # the exact commands
    for this host
```

If you took Tier 2 and want the orphaned GPU-model index gone as well:


```
lumen purge /path/to/repo          # removes ALL indexes for that
    project, both models
lumen index /path/to/repo          # rebuild
```

`lumen purge` with **no arguments** wipes every index for every project on the machine. Pass the path.

10. What the setup wizard does, and deliberately does not do

`scripts/setup-agents-wizard.sh` → `verify_lumen` will, when the backend is reachable:

- send a real embedding round-trip and require a vector back (test A33);
- name the wedged-NaN state explicitly rather than reporting a generic failure (test A34);
- ask `/api/ps` whether the embedding model is GPU-resident, read ollama's own inference compute line for the library it actually chose, and **warn with a pointer to this runbook** when that library is Vulkan (tests A35–A41).

It deliberately does **not**:

- change `LUMEN_EMBED_MODEL` — that would orphan your index (§4);
- write `/etc/sysconfig/ollama` — that needs root;
- restart any service — that would interrupt a running index without asking;
- use `sudo` anywhere in this path;
- **invoke** `scripts/ollama-vulkan-remediation.sh`. It names it; it never runs it.

When it cannot read the journal, it says the library is *unknown* rather than assuming. A warning you can trust is worth more than a warning that guesses; tests A37 and A38 keep it that way, and A41 keeps the check read-only.

It now also raises an ACTION REQUIRED step

When the journal (scanned from ollama's `ActiveEnterTimestamp`) reports `library=Vulkan`, the wizard's closing **ACTION REQUIRED** section carries an entry naming the exact commands, and writes them to `<project-root>/MANUAL-STEPS.md`:

```
Take the GPU out of the embedding path (asks for your password)
./scripts/ollama-vulkan-remediation.sh --check      #
diagnosis, read-only
./scripts/ollama-vulkan-remediation.sh --apply      # applies
it, then verifies
# WHY: this backend silently writes STALE DUPLICATE vectors
under HTTP 200 -
# well-formed, unit-norm, and invisible to every per-vector
check.
# Full runbook: docs/setup-agents-wizard/OLLAMA-
REMEDATION.md
```

On a terminal the wizard then pauses for Enter so the list cannot scroll past unread (WIZARD_NONINTERACTIVE=1 skips the pause). The entry disappears on the next run once the journal reports library=cpu — the detection is stateful, not a checklist. See [ACTION-REQUIRED.md](#).

Test A41 is what keeps the privileged commands *out* of the wizard: applying the fix is the remediation script's job, behind an explicit subcommand.

Appendix A: the state of this host AFTER remediation

Read on 2026-08-27, not assumed. Reproduce with `./scripts/ollama-vulkan-remediation.sh --check`.

```
/etc/sysconfig/ollama          GGML_VK_VISIBLE_DEVICES=-1
(single bare line: written by --apply)
inference compute ... library=cpu
(no Vulkan device line at all)
ollama ps                      ordis/jina-embeddings-v2-base-code:latest  345
MB  100% CPU  ctx 8192
journalctl -k --since "<ollama ActiveEnterTimestamp>"
                        0 x "Fence expiration time out" / "GPU HANG"
journalctl -k --since "24 hours ago"
                        520 lines – ALL pre-remediation history
batch probe                 32 chunks / 18,576 chars -> 32 distinct
vectors, 5 runs of 5
```

The index itself was rebuilt afterwards; see §9 and [OPERATIONAL-SCRIPTS.md](#).

Appendix B: the state of this host when the runbook was written

Everything below was read, not assumed. This is the **pre-remediation** snapshot, kept so the before/after in §0 is checkable. Reproduce any line with the commands in §2.

```
ollama version is 0.23.4                                (ollama-0.23.4-
alt1.x86_64, newest in Sisyphus)
inference compute ... library=Vulkan name=Vulkan0
                        description="Intel(R) Iris(R) Xe Graphics
(TGL GT2)" type=iGPU
ollama ps          ordis/jina-embeddings-v2-base-code:latest    396
MB    100% GPU    ctx 8192
/api/ps            "size_vram": 396324864                        (== size: fully
resident on the GPU)
journalctl -k --since "24 hours ago"
                        551 x "Fence expiration time out"
                        3 x "GPU HANG"
systemctl show ollama -p EnvironmentFiles
                        /etc/sysconfig/ollama (ignore_errors=yes)
<- referenced, absent
sudo -n true        -> "sudo: a password is required"
<- no passwordless sudo
ollama list         -> jina-embeddings-code-cpu:latest          (Tier 2 tag
present, PARAMETER num_gpu 0)
                        ordis/jina-embeddings-v2-base-code:latest
index               ~/.local/share/lumen/<hash>/index.db  113 MiB,
2413 files tracked,
                        embedding_model=ordis/jina-embeddings-v2-base-
code
```